



AKADEMIA GÓRNICZO-HUTNICZA IM. STANISŁAWA STASZICA W KRAKOWIE

Wydział Informatyki

Środowiska Udostępniania Usług

Grupa 1 - czwartek 8:00

Grupa 6 - piątek 15:00

IRM-G: Grafana IRM - case study

Kinga Bunkowska
Michał Mróz
Maciej Nowakowski
Albert Pęciak

Kraków, 2026

Spis treści

1	Wprowadzenie	3
2	Podstawy teoretyczne i stos technologiczny	4
2.1	Podstawy teoretyczne	4
2.2	Stos technologiczny	4
3	Opis studium przypadku	6
4	Architektura rozwiązania	7
5	Dokładna architektura rozwiązania	9
5.1	Aplikacja	9
5.2	Gromadzenie danych telemetrycznych	10
5.3	Warstwa obserwowalności i alertowania (Grafana IRM)	10
5.4	Server MCP	11
5.5	Warstwa integracji i powiadomień	11
5.6	Przepływ danych w systemie	12
6	Opis konfiguracji środowiska	14
6.1	Aplikacja	14
6.2	MCP	14
6.3	Grafana Cloud	15
6.3.1	Zespoły i harmonogramy	15
6.3.2	Powiadomienia	16
6.3.3	Ścieżki eskalacji	20
7	Metoda instalacji	22
7.1	Instalacja aplikacji	22
7.2	Uruchamianie przykładowych awarii	22
8	Przygotowanie demo	24
9	Opis demo	25
10	Podsumowanie i wnioski	31

Rozdział 1

Wprowadzenie

Współczesne systemy informatyczne charakteryzują się dużą złożonością, dlatego zapewnienie ich niezawodności stanowi istotne wyzwanie. Przerwy usług mogą powodować poważne konsekwencje dla biznesu, takie jak straty finansowe, spadek reputacji czy utrata klientów.

Nie da się zapobiec każdej awarii. Część incydentów nie wynika bezpośrednio z błędów samej aplikacji, lecz może być skutkiem braku dostępności zewnętrznych usług, opóźnień w procesach zależnych lub losowych błędów wynikających z dużego natężenia ruchu sieciowego. Często występującym problemem są również awarie kaskadowe, w których niedostępność jednego kluczowego komponentu powoduje nieprawidłowe działanie lub całkowitą awarię usług zależnych. W takich sytuacjach kluczowe znaczenie ma szybkie wykrycie incyduentu oraz jego sprawne usunięcie, aby zminimalizować wpływ na użytkowników i procesy biznesowe.

W odpowiedzi na te wyzwania wdrażane są zautomatyzowane systemy zarządzania incydentami (Incident Response Management – IRM), które integrują procesy wykrywania, klasyfikacji oraz powiadamiania o incydentach. Jednym z takich narzędzi jest Grafana Labs, które łączy mechanizmy monitorowania, alertowania, zarządzania dyżurami oraz obsługi incydentów w ramach jednego, spójnego środowiska.

Rozdział 2

Podstawy teoretyczne i stos technologiczny

2.1. Podstawy teoretyczne

Grafana Labs Incident Response Management (IRM) to moduł dostępny w ramach platformy Grafana Cloud, stanowiący zintegrowane środowisko do zarządzania incydentami w systemach informatycznych. Rozwiązanie to łączy w sobie funkcjonalności monitorowania, alertowania oraz koordynacji reakcji na zdarzenia krytyczne, umożliwiając kompleksowe podejście do obsługi incydentów w środowiskach produkcyjnych.

Jednym z kluczowych elementów systemu jest integracja z modułem Grafana Alerting, który odpowiada za wykrywanie nieprawidłowości i generowanie alertów na podstawie zdefiniowanych reguł monitoringu. Dodatkowo platforma może wykorzystywać mechanizmy analizy danych oparte na uczeniu maszynowym (ML), które wspierają identyfikację anomalii w logach oraz pozwalają na wykrywanie nietypowych wzorców zachowań systemu. Dzięki temu możliwe jest wcześniejsze wykrywanie potencjalnych problemów, zanim zostaną one zauważone przez użytkowników końcowych.

Grafana IRM automatyzuje zarządzanie procesem reagowania na incydenty poprzez automatyczne powiadamianie odpowiednich zespołów deweloperskich i operacyjnych. System wspiera przekazywanie alertów oraz ich eskalację, co zapewnia szybkie przypisanie odpowiedzialności i skrócenie czasu reakcji, bez potrzeby utrzymywania struktury wsparcia, która robiłaby to ręcznie. Wspierane jest też zarządzanie dyżurami (on-call), która pozwala członkom zespołów na rotacyjne pełnienie dyżurów, co umożliwia równomierne rozłożenie obciążenia operacyjnego i zapewnia ciągłą dostępność osób odpowiedzialnych za reakcję na incydenty.

Dodatkowo platforma oferuje narzędzia do analizy historycznych incydentów, co umożliwia identyfikację ich przyczyn źródłowych (root cause analysis) oraz wdrażanie działań zapobiegawczych. Dzięki temu organizacje mogą systematycznie ograniczać ryzyko występowania podobnych problemów w przyszłości oraz poprawiać ogólną stabilność systemów.

2.2. Stos technologiczny

Głównym komponentem technologicznym projektu jest Grafana IRM, która odpowiada za zarządzanie incydentami oraz wizualizację stanu systemu. W celu zbierania metryk wykorzystane zostanie OpenTelemetry, natomiast do ich przetwarzania wykorzystany zostanie Prometheus. Grafana Alerting umożliwi definiowanie i obsługę alertów. Wykorzystane technologie pozwalają na kompleksowe monitorowanie i zarządzanie incydentami aplikacji. W projekcie używany jest komunikator Discord - będzie to medium, na które przesyłane będą komunikaty

o nieprawidłowym działaniu aplikacji. Udostępniony serwer MCP pozwala na zmianę konfigurację alertów przy pomocy języka naturalnego przy pomocy wybranego modelu typu LLM. Sam projekt jest kompatybilny z wieloma dostępnymi modelami, ale był projektowany i testowany przy pomocy Claude Code od firmy Anthropic z użyciem modelu Sonnet 4.6. Głównym powodem wybrania tego modelu był fakt, że jest on stosunkowo tani w porównaniu do innych modeli od tej firmy, a dostarczane narzędzie Claude Code umożliwia stworzenie rozwiązania kompatybilnego z lokalnym terminalem.

Projekt będzie uruchamiany przy wykorzystaniu platformy Kubernetes z narzędziem Istio, ułatwiającym dołączenie programów do obserwacji takich jak OpenTelemetry czy Prometheus, jak również umożliwi symulowanie awarii serwisów w aplikacji. Jako platformę hostującą projekt wykorzystany zostanie Elastic Kubernetes Service (EKS) udostępniany przez firmę Amazon.

Rozdział 3

Opis studium przypadku

Planujemy wykorzystać zestaw mniejszych usług, które zostaną zintegrowane w ramach jednej aplikacji webowej wspierającej planowanie podróży wakacyjnych. Aplikacja będzie pełnić rolę centralnego systemu agregującego dane z trzech zewnętrznych źródeł: usługi mapowej, kalendarza zawierającego informacje o weekendach i dniach ustawowo wolnych oraz serwisu pogodowego dostarczającego prognozy.

Na podstawie zebranych danych aplikacja będzie analizować potencjalne okresy wolne od pracy i zestawiać je z prognozowaną pogodą. Wyniki tej analizy będą prezentowane na mapie Polski poprzez wskazanie regionów, w których w wybranym okresie można spodziewać się korzystnych warunków pogodowych.

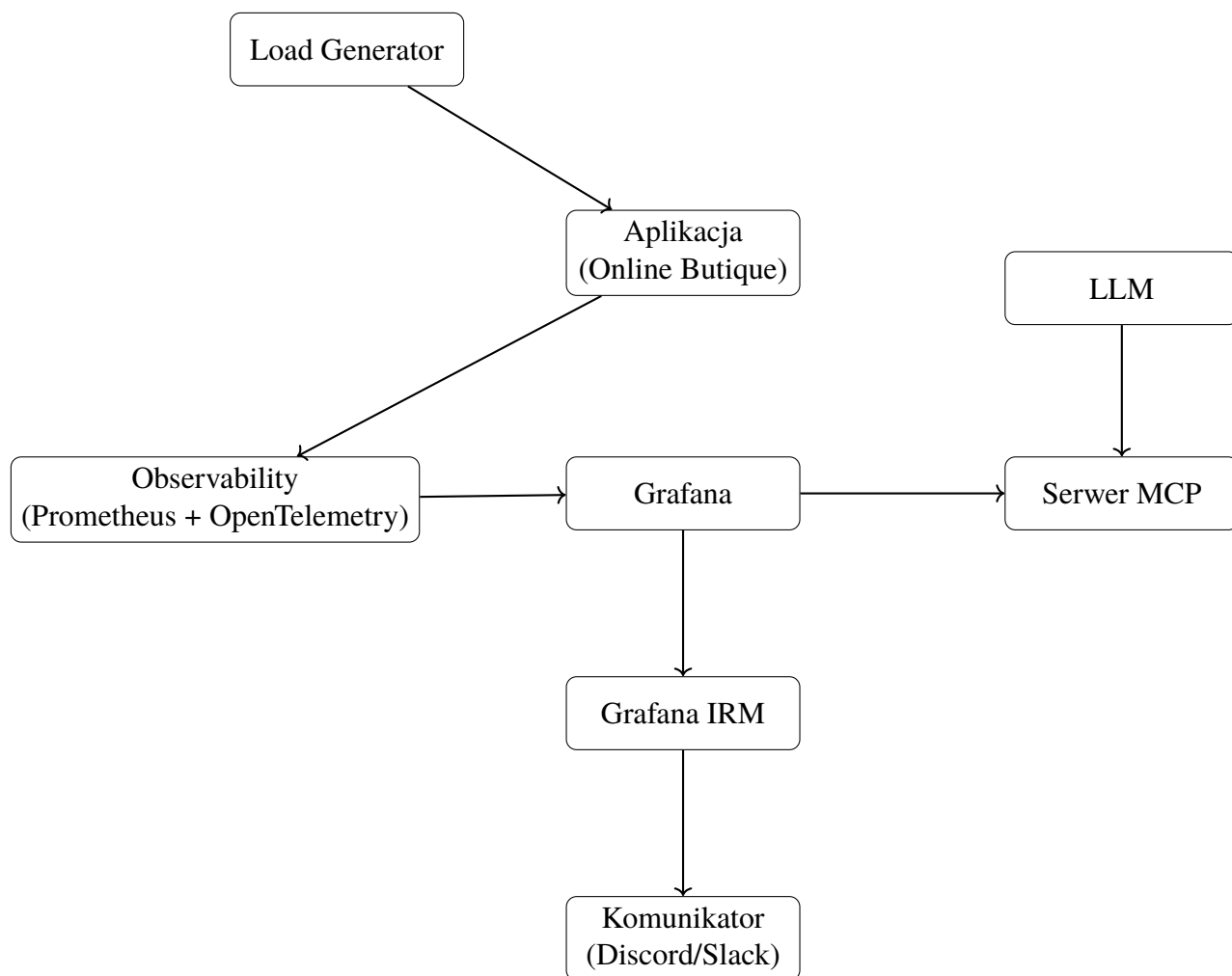
W ramach studium przypadku przygotujemy zestaw scenariuszy symulujących różne problemy związane z działaniem zewnętrznych usług (np. brak odpowiedzi API, opóźnienia lub błędne dane). System monitorowania oparty na Grafanie będzie zbierał metryki i logi z aplikacji, natomiast użytkownik otrzyma możliwość konfiguracji reguł alertowania przy wykorzystaniu warstwy LLM. W połączeniu z odpowiednio zintegrowanym i skonfigurowanym serwerem MCP pozwoli to na bardziej intuicyjne określanie warunków powiadomień.

W przypadku wykrycia incydentu Grafana ORM będzie odpowiedzialna za generowanie i dystrybucję alertów do wybranego komunikatora (np. Discord lub Slack).

Rozdział 4

Architektura rozwiązania

Architektura rozwiązania opisuje przepływ danych i zdarzeń w systemie, który umożliwia wykrywanie, analizę oraz obsługę niepożądanych zdarzeń. Informacje o incydentach są przetwarzane i przekazywane do odpowiednich osób za pośrednictwem właściwie dobranych kanałów komunikacyjnych.



Aplikacja stanowi monitorowany obiekt, którego zadaniem jest symulowanie niepożądanych zdarzeń. Jest to prosta aplikacja webowa oparta o mikroserwisy pełniąca rolę systemu testowego

- Online Butique. Generuje ona metryki (np. latency, error rate, CPU usage), logi oraz zdarzenia błędów (np. HTTP 5xx, wyjątki). Aplikacja udostępnia dane telemetryczne w postaci metryk poprzez endpoint zgodny z formatem Prometheus lub z wykorzystaniem OpenTelemetry, a opcjonalnie także logi i zdarzenia.

Warstwa observability oparta na narzędziu Prometheus odpowiada za cykliczne pobieranie metryk z aplikacji. System ten zajmuje się przechowywaniem danych w postaci szeregów czasowych oraz ich udostępnianiem do dalszej analizy i wizualizacji.

Grafana pełni rolę centralnego punktu wizualizacji i analizy danych. Pobiera dane z Prometheusa za pośrednictwem jego API i umożliwia tworzenie interaktywnych dashboardów prezentujących stan systemu w czasie rzeczywistym. Oprócz wizualizacji Grafana odpowiada również za konfigurację mechanizmów alertowania (Grafana Alerting). W przypadku wykrycia nieprawidłowości generowane alerty są przekazywane do systemu zarządzania incydentami.

Model językowy (LLM), np. dostarczany przez OpenAI lub Anthropic, wspiera proces konfiguracji środowiska Grafany. Ma umożliwić konfigurację harmonogramu dyżurów oraz reguł alertów na podstawie poleceń użytkownika, co znacząco powinno uproszczyć i przyspieszyć zarządzanie systemem.

Zarządzanie incydentami realizowane jest przez moduł Grafana Cloud IRM, który rozszerza funkcjonalność Grafany o operacyjne zarządzanie zdarzeniami. IRM agreguje alerty w incydenty, umożliwia ich klasyfikację, przypisywanie do osób pełniących dyżur, a także obsługuje eskalację i śledzenie przebiegu incydentu w czasie. System odpowiada również za routing powiadomień do odpowiednich odbiorców.

Serwer MCP (Model Context Protocol) to warstwa pośrednia udostępniająca dane i narzędzia LLM-owi.

Komunikator, taki jak Discord lub dedykowana aplikacja Grafana IRM, stanowi końcowy element systemu. To właśnie za jego pośrednictwem przekazywane są powiadomienia o incydentach do wyznaczonych osób, co umożliwia szybką reakcję i podjęcie odpowiednich działań.

Rozdział 5

Dokładna architektura rozwiązania

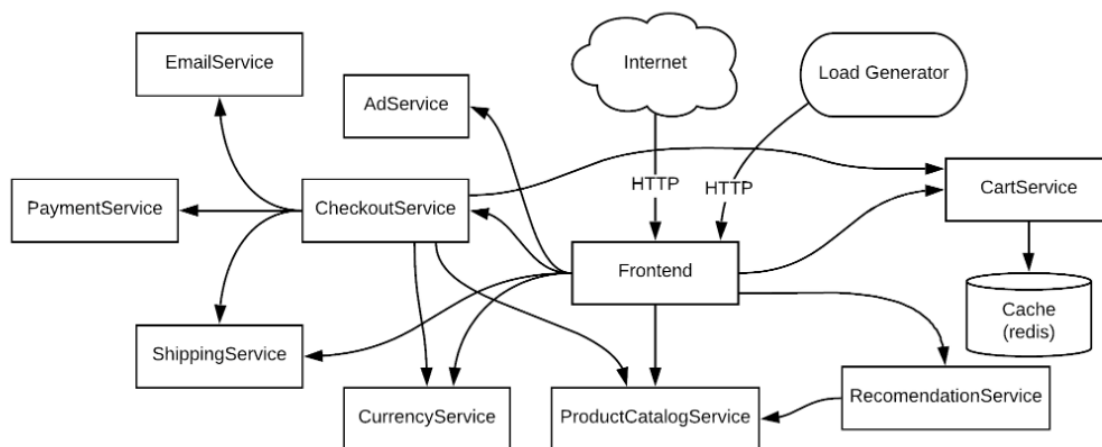
5.1. Aplikacja

System oparty jest na referencyjnej aplikacji Online Boutique wdrożonej w klastrze Kubernetes. Architektura ma charakter mikroserwisowy — każda usługa działa jako niezależny kontener i implementuje wyodrębnioną funkcjonalność biznesową, taką jak obsługa koszyka, katalog produktów, przetwarzanie zamówień czy realizacja płatności.

Kluczowe cechy warstwy aplikacyjnej:

- Komunikacja między usługami realizowana jest z wykorzystaniem protokołów HTTP, gRPC oraz TCP,
- Każda usługa eksponuje endpoint metryk zgodny z formatem Prometheus,
- Komponent *loadgenerator* symuluje realistyczny ruch użytkowników, generując obciążenie systemu i umożliwiając obserwację zachowania aplikacji w warunkach produkcyjnych.

Warstwa ta stanowi główne źródło danych telemetrycznych wykorzystywanych w całym systemie obserwowalności.



Rysunek 5.1: Architektura aplikacji Online Boutique

5.2. Gromadzenie danych telemetrycznych

Warstwa telemetryczna oparta jest na systemie Prometheus, który pełni rolę centralnego systemu zbierania metryk oraz lekkiej bazy danych szeregów czasowych (TSDB).

Każda mikrousługa udostępnia zestaw standardowych metryk, obejmujących między innymi:

- histogramy czasu odpowiedzi (request latency),
- liczniki liczby żądań (request rate),
- wskaźniki błędów HTTP (4xx/5xx error rate),
- metryki wykorzystania zasobów systemowych (CPU, pamięć).

Prometheus działa w modelu *pull-based scraping*, okresowo pobierając metryki z endpointów usług. Zebrane dane są przechowywane jako szeregi czasowe, co umożliwia zarówno analizę historyczną, jak i bieżące zapytania diagnostyczne.

Do analizy danych oraz definiowania reguł alertowania wykorzystywany jest język PromQL, który stanowi podstawę zapytań zarówno w Grafanie, jak i w systemie alertingu.

5.3. Warstwa obserwowalności i alertowania (Grafana IRM)

Warstwa obserwowalności oparta jest na platformie Grafana Cloud, w szczególności na modułach Grafana Alerting oraz Grafana IRM (Incident Response Management).

Warstwa ta realizuje trzy główne funkcje:

- wizualizację danych telemetrycznych w postaci dashboardów (metryki usługowe i klastrów),
- korelację sygnałów z różnych źródeł (metryki, logi, alerty),
- automatyczną detekcję incydentów oraz zarządzanie ich cyklem życia.

Reguły alertowe definiowane są na podstawie warunków zbliżonych do SLO/SLI, takich jak:

- wzrost opóźnień odpowiedzi powyżej zdefiniowanego progu,
- zwiększony współczynnik błędów w określonym oknie czasowym,
- brak danych telemetrycznych wskazujący na potencjalną awarię usługi.

Po spełnieniu warunków alertu system Grafana generuje zdarzenie incydentu zawierające ustrukturyzowane dane, w tym:

- identyfikator usługi źródłowej,
- wartości metryk w momencie wyzwolenia alertu,
- kontekst czasowy (okno ewaluacji),
- poziom krytyczności (severity).

5.4. Serwer MCP

Serwer MCP (Model Context Protocol) pełni rolę warstwy integracyjnej pomiędzy modelem językowym (LLM) a API Grafana OnCall w ekosystemie Grafana Labs, działającym w ramach platformy Grafana Cloud. Jego głównym celem jest umożliwienie bezpiecznej, kontrolowanej i deterministycznej modyfikacji konfiguracji dyżurów (on-call), harmonogramów oraz operacji rotacyjnych na użytkownikach.

Komunikacja z API obejmuje w szczególności:

- zarządzanie użytkownikami OnCall (wyszukiwanie, filtrowanie),
- odczyt i modyfikację harmonogramów dyżurów (schedules),
- odczyt i aktualizację shiftów (on-call shifts),
- operacje na rotacjach użytkowników (replace/swap w obrębie shiftów),
- zarządzanie i inspekcję żądań zamiany dyżurów (shift swap requests),
- tworzenie, aktualizację i usuwanie requestów swapów z obsługą trybu dry-run.

LLM generuje ustrukturyzowane intencje operacyjne, takie jak identyfikacja użytkowników (ID/username/email), operacje swap/replace w ramach shiftów lub wielu shiftów, definicje zakresów czasowych (lokalnych i UTC), operacje planowania zmian w harmonogramach.

MCP odpowiada za ich walidację, normalizację oraz translację na poprawne payloady API. Kluczowym elementem jest sanitizacja danych wejściowych, polegająca na usuwaniu pól systemowych (np. id, created_at, updated_at), które nie mogą być modyfikowane bezpośrednio przez klienta API.

5.5. Warstwa integracji i powiadomień

System integruje się z zewnętrznymi kanałami komunikacji, takimi jak Discord oraz SMS, które pełnią rolę kanałów powiadomień operacyjnych. Dodatkowo wykorzystywana jest aplikacja Grafana jako interfejs do zarządzania incydentami.

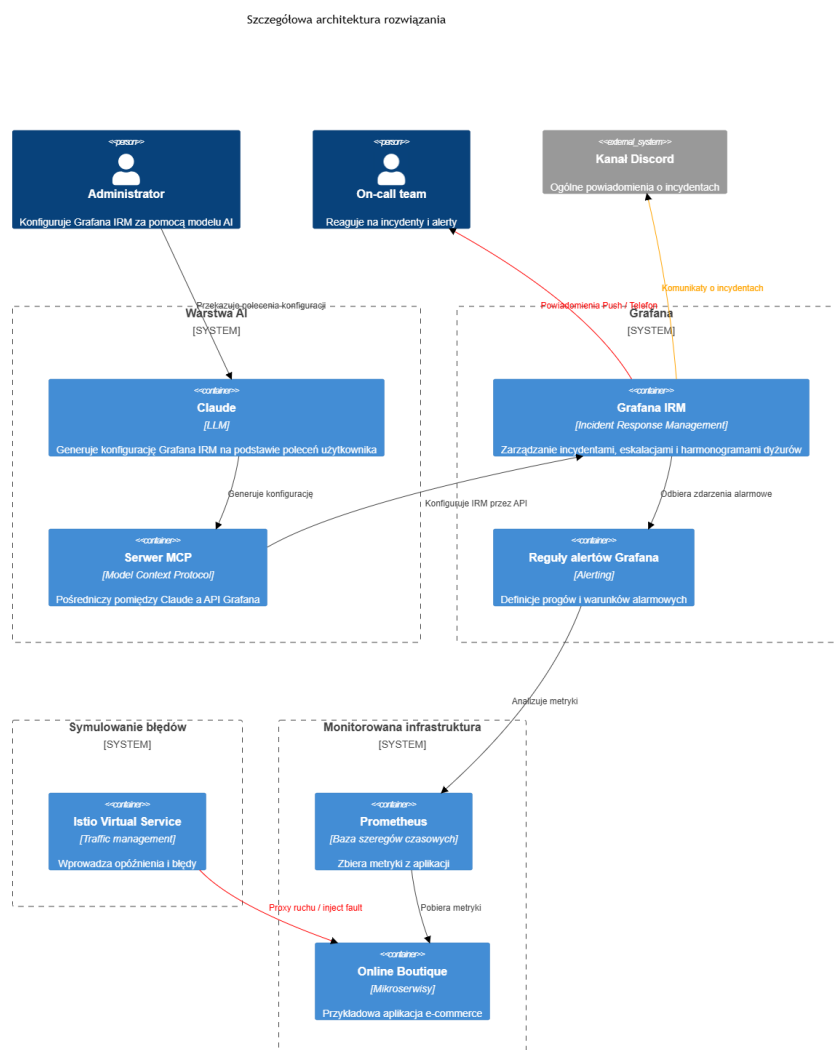
W przypadku wykrycia incydu:

- Grafana Alerting generuje zdarzenie na podstawie reguły SLO/SLI,
- Grafana IRM przekształca alert w incydent operacyjny i przypisuje jego kontekst (usługa, severity, czas, metryki),
- MCP może dynamicznie wzbogacić incydent o dane konfiguracyjne (np. przypisany on-call, mapowanie zespołu),
- zdarzenie jest dystrybuowane do odpowiednich kanałów komunikacyjnych zgodnie z polityką eskalacji.

Dobór odbiorców powiadomień zależy od poziomu severity oraz przypisania zespołu on-call w Grafana IRM.

5.6. Przepływ danych w systemie

Przepływ danych w systemie obejmuje cały cykl obsługi incydentu – od wygenerowania danych telemetrycznych przez aplikację, poprzez ich analizę i wykrycie nieprawidłowości, aż do dostarczenia powiadomienia odpowiednim osobom. Poszczególne komponenty architektury współpracują ze sobą, tworząc zautomatyzowany proces monitorowania, alertowania oraz zarządzania incydentami. Schemat przedstawiający zależności pomiędzy elementami systemu zaprezentowano na rysunku poniżej.



Całościowy przepływ danych w systemie przebiega następująco:

- Mikroserwisy generują metryki aplikacyjne i systemowe w czasie rzeczywistym,
- Prometheus cyklicznie scrapuje endpointy metryk i zapisuje dane jako szeregi czasowe,
- Grafana analizuje dane przy użyciu zapytań PromQL i ocenia warunki reguł alertowych,
- Po spełnieniu warunków alertu generowane jest zdarzenie,

- Grafana IRM przekształca alert w incydent i nadaje mu kontekst operacyjny,
- Serwer MCP może w trakcie cyklu życia incyduentu modyfikować konfigurację (np. eskalacje, on-call, reguły),
- Powiadomienia są dystrybuowane do kanałów zewnętrznych (Discord, SMS, Grafana UI).

Rozdział 6

Opis konfiguracji środowiska

6.1. Aplikacja

Program przygotowany jest do uruchomienia przy użyciu Kubernetes. Na potrzeby projektu wykorzystaliśmy w tym celu Elastic Kubernetes Service udostępniany przez firmę Amazon. Przygotowany program wymaga przynajmniej 3 instancji typu t3.medium by móc poprawnie działać. W celu wykorzystania usługi EKS należy zainstalować AWS CLI zgodnie z instrukcją producenta: (<https://docs.aws.amazon.com/cli/latest/userguide/getting-started-install.html#getting-started-install-instructions>). Dodatkowymi narzędziami wymaganymi do uruchomienia aplikacji są kubectl - narzędzie wiersza poleceń umożliwiające konfigurację kubernetes (<https://kubernetes.io/docs/tasks/tools/#kubectl>), oraz istio (<https://istio.io/latest/docs/setup/additional-setup/download-istio-release/>)

Po utworzeniu klastra i zainstalowaniu wymaganych programów należy pobrać token uwierzytelniający AWS i umieścić go w pliku `~/.aws/credentials`. Następnie należy skonfigurować narzędzie aws cli oraz połączenie z serwisem EKS, przy wykorzystaniu następujących poleceń:

```
1 aws configure
2 aws configure set default.region us-east-1
3 aws configure set default.output table
4
5 aws eks describe-cluster --region us-east-1 --name <your cluster> /
6   --query cluster.status
7 aws eks --region us-east-1 update-kubeconfig --name <your cluster>
```

Listing 6.1: Konfiguracja aws cli

W powyższych (6.1) poleceniach `<your cluster>` należy zastąpić nazwą stworzonego klastra Kubernetes.

Po wykonaniu powyższych czynności należy pobrać repozytorium z projektem z adresu <https://github.com/AlbertPec/IRM-with-Grafana>, a następnie skopiować plik `env.example`. Pola `GF_IRM_WEBHOOK_URL` oraz `GF_DISCORD_WEBHOOK_URL` należy uzupełnić o adresy url wykorzystywane przy komunikacji z Grafana Cloud.

6.2. MCP

Konfiguracja środowiska dla serwera MCP zależy od modelu LLM, do którego chcemy podłączyć Grafanę. W ramach tego projektu postanowiliśmy wykorzystać Claude Code, udo-

stępniany przez firmę Anthropic. Głównym powodem wyboru tego rozwiązania była wygodna możliwość wydawania poleceń do Grafany bezpośrednio z lokalnego terminala.

Na samym początku należy zainstalować Claude Code na lokalnym urządzeniu, zgodnie z instrukcją dostępną na stronie producenta: <https://code.claude.com/docs/en/quickstart>.

Następnie, w celu skonfigurowania środowiska, należy pobrać repozytorium z GitHuba: <https://github.com/AlbertPec/IRM-with-Grafana>. Po pobraniu repozytorium trzeba skopiować zawartość pliku `example.mcp.json` do pliku `.mcp.json`.

Pola `<GRAFANA_URL>` oraz `<GRAFANA_STACK_URL>` należy uzupełnić adresem URL Grafany przypisanym do naszej organizacji. Wartość `<GRAFANA_SERVICE_ACCOUNT_TOKEN>` należy zastąpić kluczem uzyskanym po utworzeniu konta serwisowego pod adresem `<GRAFANA_URL>/org/serviceaccounts`. Utworzonemu kontu należy przyznać następujące uprawnienia w celu umożliwienia wykorzystania wszystkich operacji oferowanych przez MCP: `Admin`, `Organizations:Reader`, `Teams:Writer`, `Teams:Reader`, `Teams:Permission reader` oraz `Teams:Permission writer`.

Po wykonaniu tych kroków można uruchomić agenta z folderu, w którym znajduje się plik `.mcp.json`. Agent będzie miał wtedy dostęp do Grafany i będzie mógł korzystać ze wszystkich lokalnie przygotowanych komend serwera MCP.

6.3. Grafana Cloud

W celu przygotowania wersji demo została stworzona w Grafanie domena <https://mnowak.grafana.net/>. Została ona połączona z aplikacją wdrożoną na AWS, aby umożliwić śledzenie jej aktualnego stanu.

6.3.1. Zespoły i harmonogramy

Zostało stworzone konto w Grafanie dla każdego członka naszego zespołu. W sekcji **Administration -> Users and access -> Teams** Grafany Cloud zostały stworzone dwa zespoły, do których zostały przydzieleni użytkownicy.

Name	Email	Members	Role
 team A		3	No roles assigned  
 team B		3	No roles assigned  

Rysunek 6.1: Zespoły w Grafanie

Każdy zespół utworzył osobny harmonogram w sekcji **Alerts & IRM -> IRM -> Schedules**.

	Type	Status	Name	On-call now	Team	
>	Web	4	A-schedule		team A	
>	Web	1	B-schedule	kbunkowska	team B	

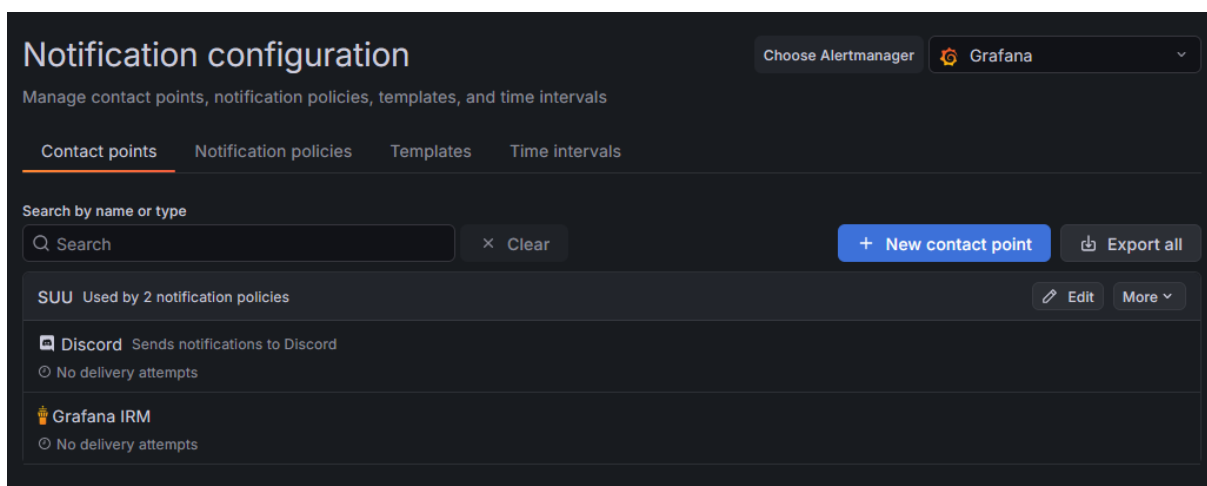
Rysunek 6.2: Harmonogramy zespołów

6.3.2. Powiadomienia

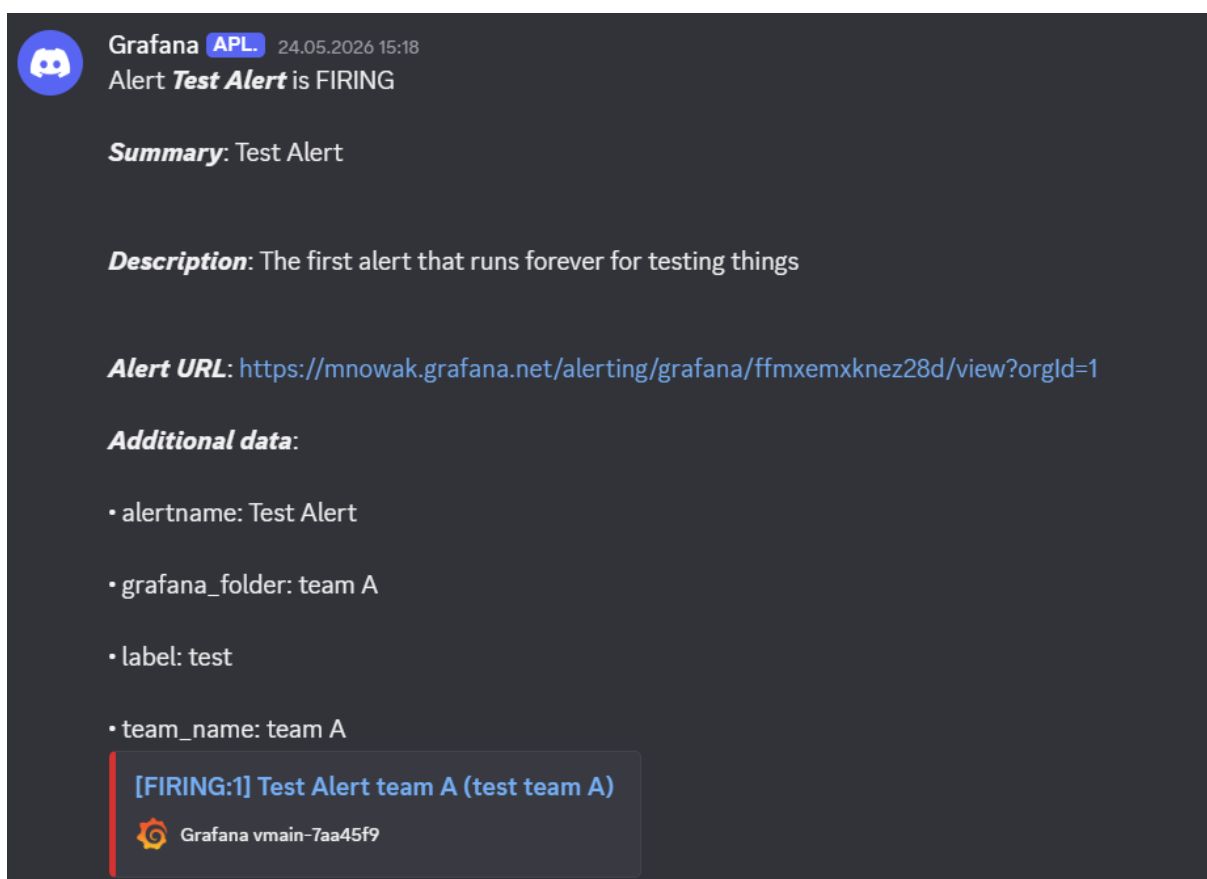
Zostały skonfigurowane sposoby różne powiadamiania o alertach i incydentach, zarówno zbiorowe (przez platformę Discord), jak i indywidualne, zależne od przypisania do harmonogramu.

Powiadomienia na platformie Discord

W sekcji **Alerts & IRM -> Alerting -> Notification configuration** został dodany **contact point** 6.3, który poza przekazywaniem alertów do Grafany IRM, przesyła wiadomość na platformę Discord 6.4.

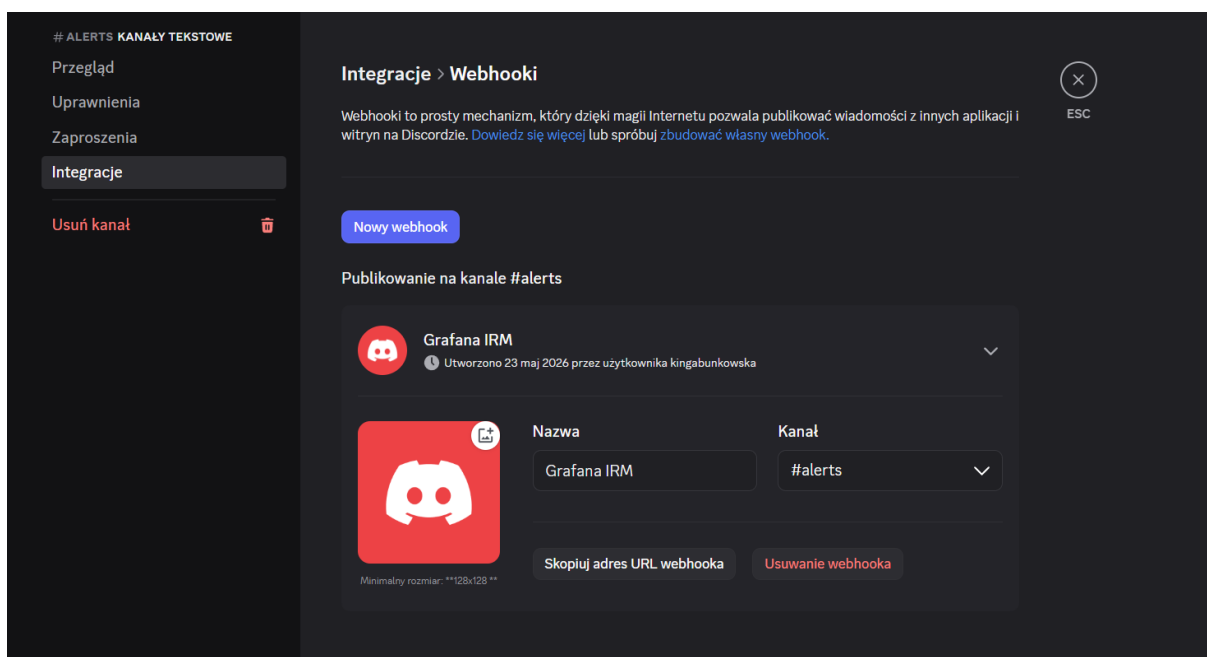


Rysunek 6.3: Przykładowe punkty kontaktu



Rysunek 6.4: Przykładowa wiadomość otrzymana na platformie Discord

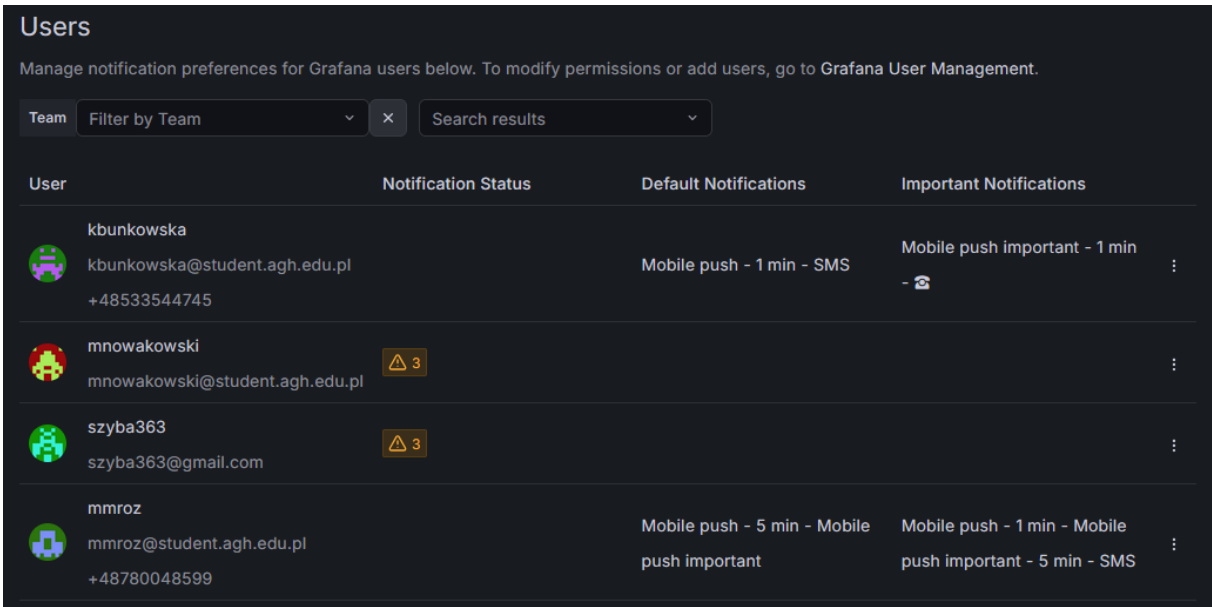
Integrację przeprowadza się poprzez podanie Grafanie url'a webhooka kanału Discord, który można otrzymać z platformy Discord z widoku serwera w **Edytuj kanał -> Integracje -> Webhooki** 6.5.



Rysunek 6.5: Stworzenie webhooka na kanał Discord

Powiadomianie użytkowników

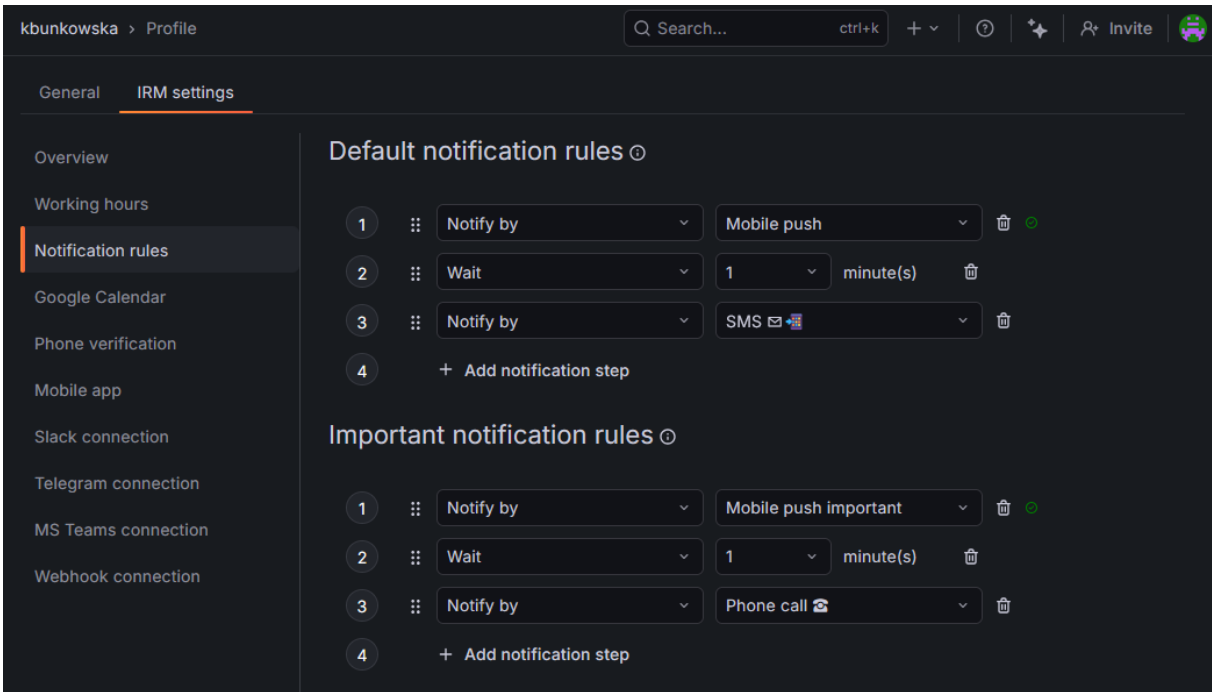
W Grafanie każdy użytkownik ma możliwość ustawienia własnych kanałów otrzymywania powiadomień 6.6. Ustawienia można zmienić w zakładce **Profile -> IRM settings -> Notification rules** 6.7. Do wyboru jest wiele opcji, np. SMS, połączenie telefoniczne, wiadomość email oraz powiadomienie push na telefon przy pomocy aplikacji mobilnej Grafany IRM.



The screenshot shows the 'Users' management page in Grafana. It lists four users with their notification settings. The first user, kbunkowska, has mobile push and SMS notifications. The second and third users, mnowakowski and szyba363, have warning icons. The fourth user, mmroz, has mobile push and SMS notifications.

User	Notification Status	Default Notifications	Important Notifications
kbunkowska kbunkowska@student.agh.edu.pl +48533544745		Mobile push - 1 min - SMS	Mobile push important - 1 min -
mnowakowski mnowakowski@student.agh.edu.pl	⚠ 3		
szyba363 szyba363@gmail.com	⚠ 3		
mmroz mmroz@student.agh.edu.pl +48780048599		Mobile push - 5 min - Mobile push important	Mobile push - 1 min - Mobile push important - 5 min - SMS

Rysunek 6.6: Powiadomienia ustawione przez użytkowników



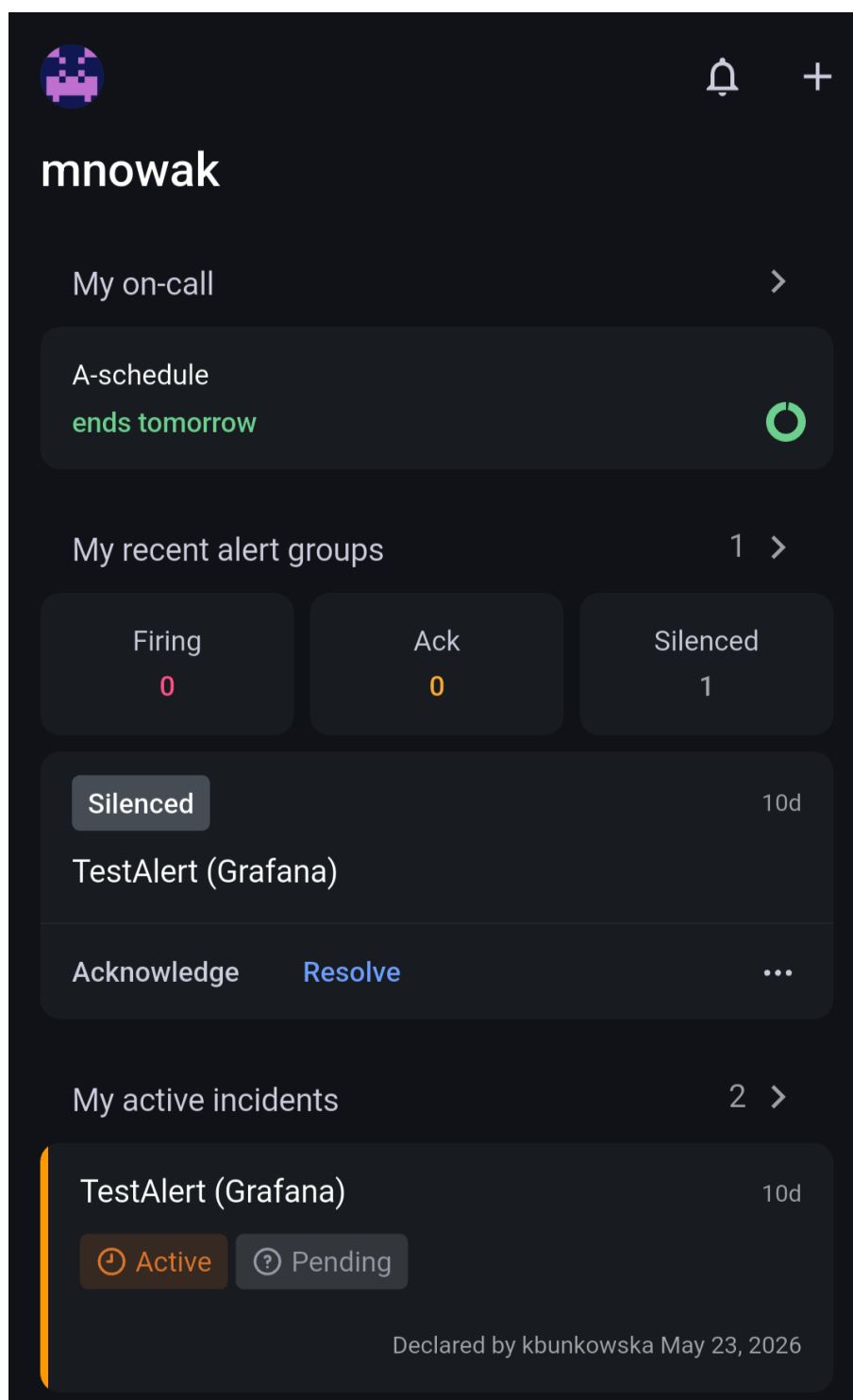
The screenshot shows the 'Notification rules' panel in the Grafana profile settings. It displays two sections: 'Default notification rules' and 'Important notification rules'. Each section has a list of rules with steps for 'Notify by', 'Wait', and 'Notify by'. The first rule in the default section is 'Mobile push' with a 1-minute wait. The second rule is 'SMS'. The first rule in the important section is 'Mobile push important' with a 1-minute wait. The second rule is 'Phone call'.

Section	Step	Notify by	Wait	Unit
Default notification rules	1	Mobile push	1	minute(s)
	3	SMS		
Important notification rules	1	Mobile push important	1	minute(s)
	3	Phone call		

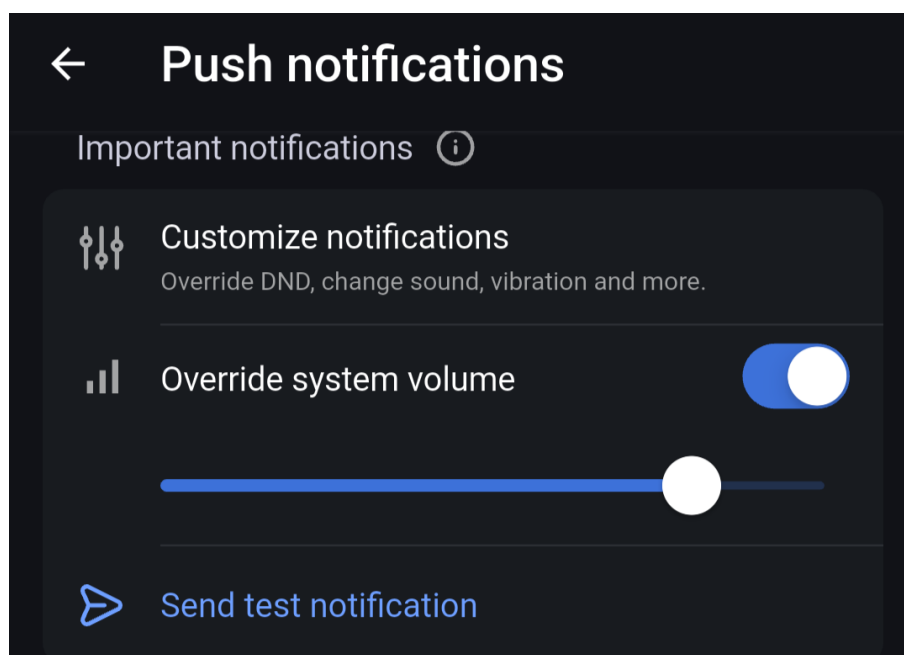
Rysunek 6.7: Panel ustawiania sposobów powiadomienia

Po pobraniu aplikacji łączy się ją do Grafany w sekcji **Profile -> IRM settings -> Mobile app** przez zeskanowanie kodu QR. W aplikacji, poza możliwością zobaczenia harmonogramów oraz stanu incydentów 6.8, można ustawić nadpisanie głośności urządzenia dla konkretnego

poziomu powiadomienia 6.9 w sekcji **Settings -> Push notifications**. W tym samym miejscu można ustawić powiadamianie o początku rotacji on-call.



Rysunek 6.8: Widok aplikacji Grafana IIRM



Rysunek 6.9: Ustawienie nadpisania głośności dla ważnych powiadomień

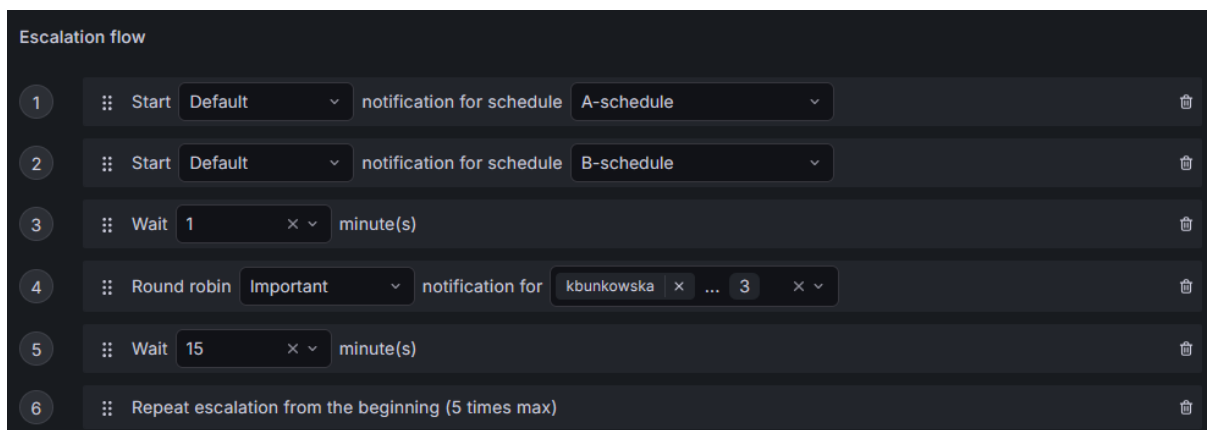
6.3.3. Ścieżki eskalacji

Zostały przygotowane dwie ścieżki eskalacji 6.10 w sekcji **Alerts IRM -> IRM -> Escalation chains**.

Escalation Chains			
Team	Filter by Team	Search results	+ New escalation chain
Name	Team	Status	
only team A	team A	2	
All teams	No team	1	

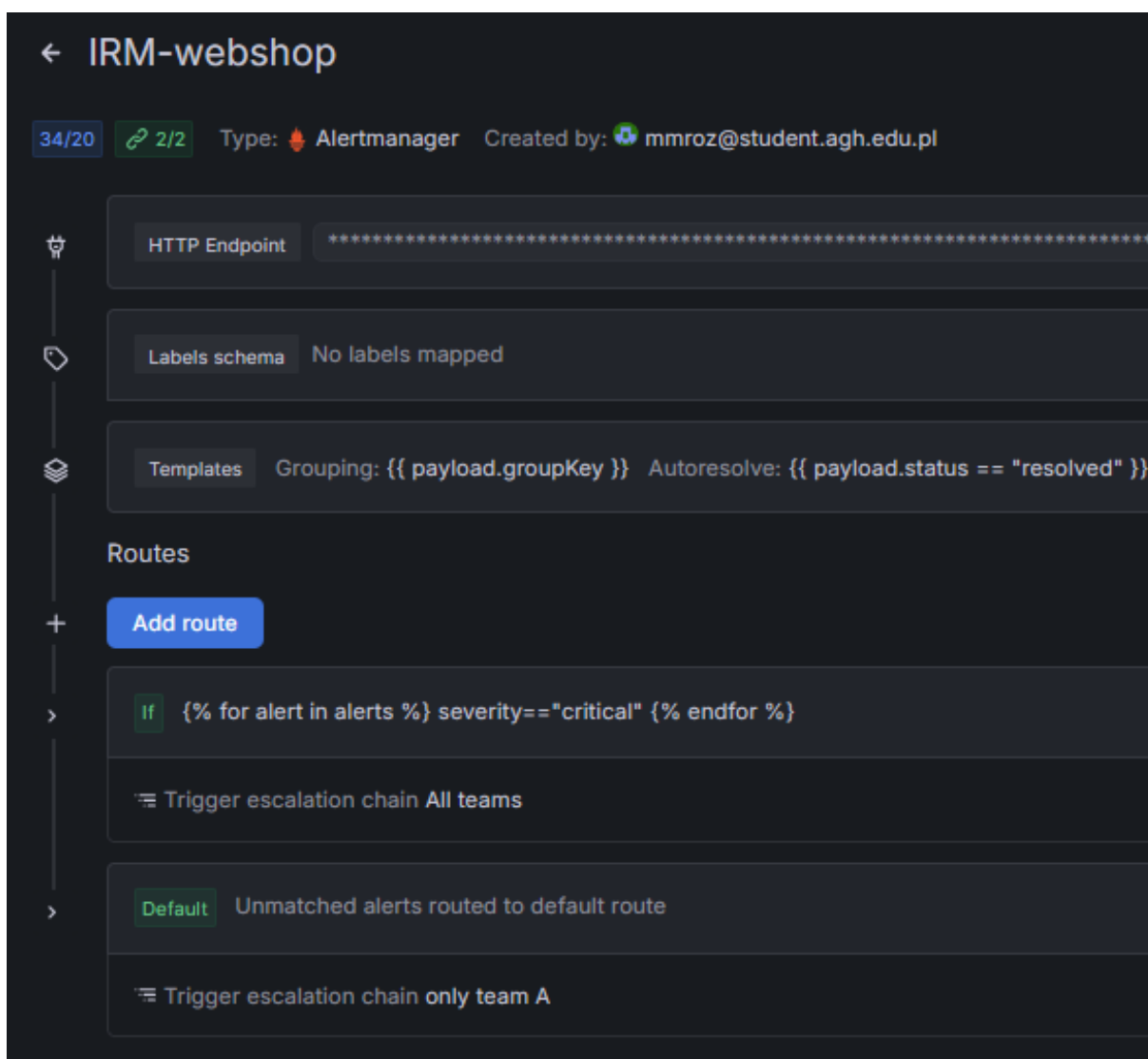
Rysunek 6.10: Przygotowane ścieżki eskalacji

Jedna z nich (All teams) prezentuje poinformowanie o alercie członków z harmonogramu obu zespołów. W razie braku potwierdzenia przez żadnego członka zespołu przez minutę, po kolei informowani będą wszyscy możliwi użytkownicy na wyższym poziomie powiadomienia.



Rysunek 6.11: Przykładowa ścieżka eskalacji

Obie ścieżki eskalacji są ustawione na **route**'ach w integracji 6.12 w sekcji **Alerts & IRM -> IRM -> Integrations**, w zależności od oznaczenia dotkliwości wybierana jest ścieżka eskalacji, która zostanie uruchomiona.



Rysunek 6.12: Przykładowa integracja z ustawionymi route'ami

Rozdział 7

Metoda instalacji

7.1. Instalacja aplikacji

Przed zainstalowaniem aplikacji należy poprawnie skonfigurować środowisko, co zostało przedstawione w rozdziale 6. Całą aplikację można uruchomić wykonując skrypt **deploy.sh**. Program ten instaluje kolejno Istio, używaną aplikację, Prometheusa oraz Grafanę. Skrypt ten dodaje również odpowiednie konfiguracje do Grafany, umożliwiające komunikację z usługą Grafana Cloud. Skrypt podaje również adres umożliwiający włączenie aplikacji, oraz komendę potrzebną do otworzenia panelu Grafany. By otrzymać adres aplikacji ręcznie można wykonać w terminalu polecenie przedstawione na listingu 7.1.

```
kubectl get svc istio-ingressgateway -n istio-system -o \
  jsonpath='{.status.loadBalancer.ingress[0].hostname}'
```

Listing 7.1: Uzyskanie adresu aplikacji

W celu uruchomienia panelu Grafany należy natomiast wykonać polecenie przedstawione na listingu 7.2.

```
istioctl dashboard grafana
```

Listing 7.2: Uruchomienie panelu Grafana

7.2. Uruchamianie przykładowych awarii

Do projektu dołączono zestaw skryptów symulujących przykładowe awarie aplikacji, dostępne w folderze *fault_scripts*. Programy te wykorzystują typ Virtual Service z Istio, modyfikując działanie wybranych części programu, wprowadzając przykładowo większe opóźnienie w komunikacji między serwisami, lub też sprawiając, że wybrane serwisy zwracają błąd zamiast poprawnego wyniku.

Aby uruchomić przykładowy skrypt należy wykonać w terminalu polecenie przedstawione na listingu 7.3, gdzie *selected_script* to wybrany skrypt.

```
kubectl apply -f fault_scripts/<selected_script>
```

Listing 7.3: Uruchomienie przykładowej awarii systemu.

W celu usunięcia awarii wystarczy wykonać polecenie przedstawione poniżej, zastępując *selected_script* z konkretną awarią którą chcemy usunąć.

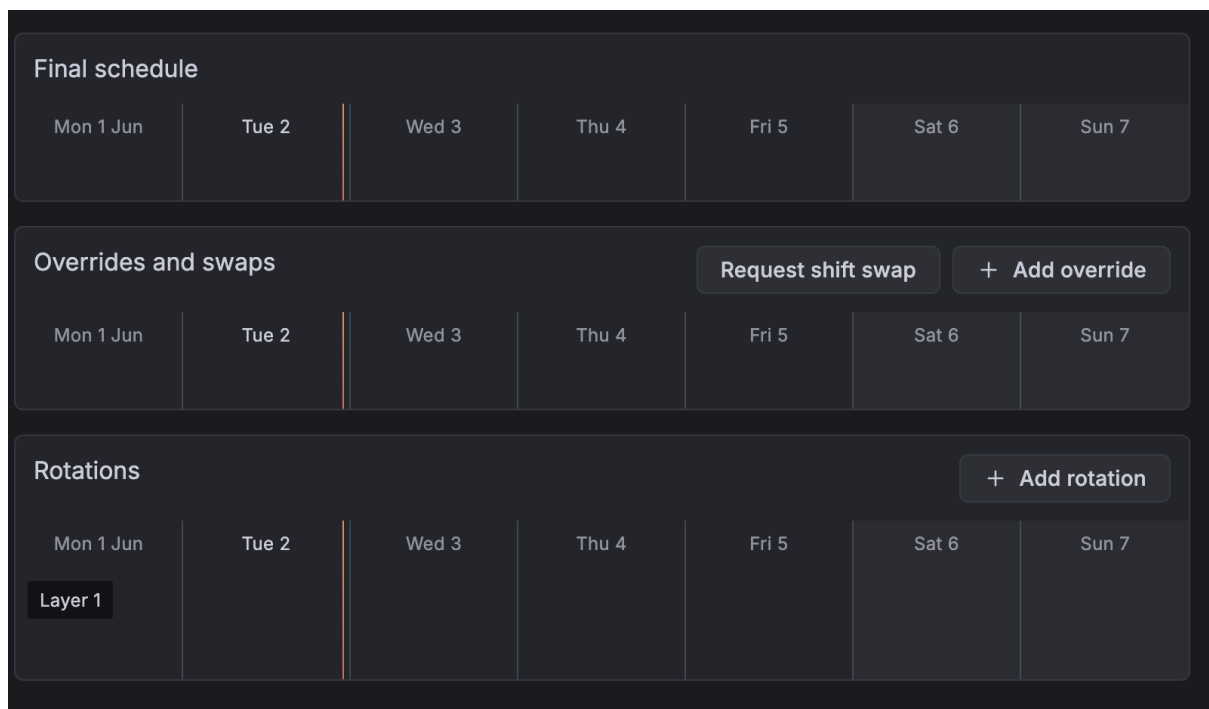
```
kubectl delete -f fault_scripts/<selected_script>
```

Listing 7.4: Usuwanie przykładowej awarii systemu.

Rozdział 8

Przygotowanie demo

Przed zaprezentowaniem możliwości MCP należało usunąć z harmonogramu B konta przypisane wcześniej do odpowiednich dni podczas testów.



Rysunek 8.1: Wyczyszczony harmonogram B

Rozdział 9

Opis demo

Na początku prezentowana jest aktualna konfiguracja Grafany oraz obowiązujące harmonogramy. Następnie wykonywane są kolejne kroki demonstracji:

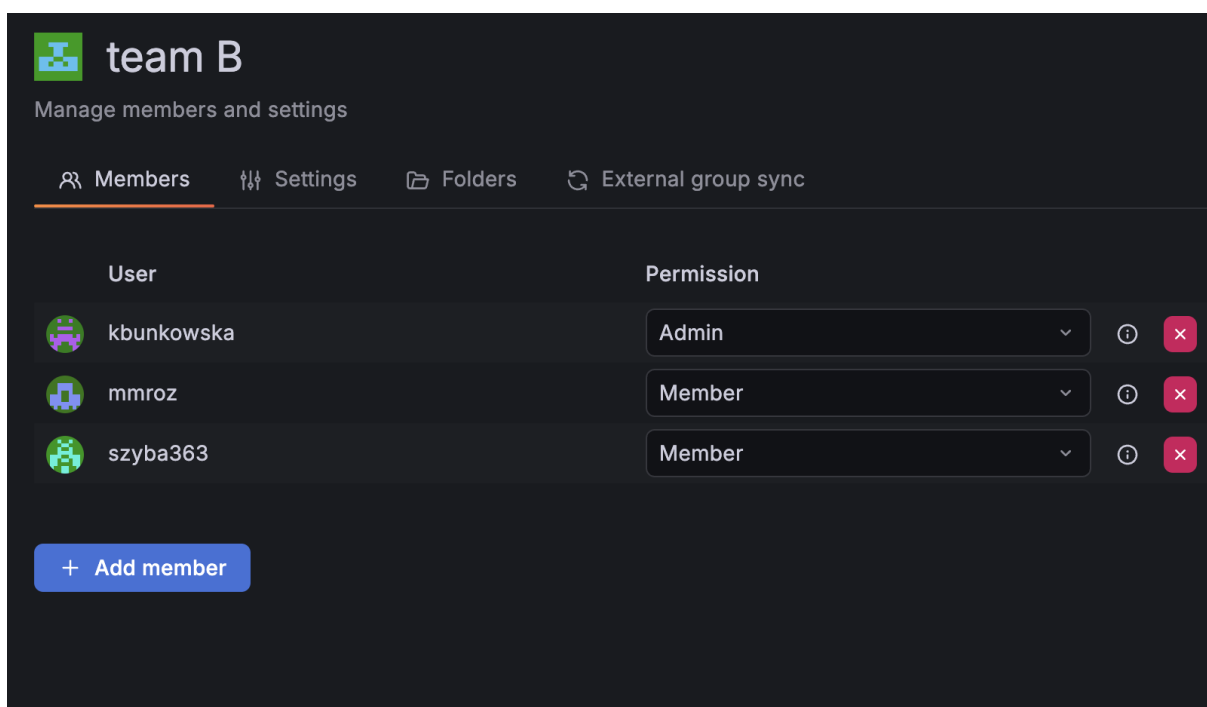
1. Opisanie aktualnej konfiguracji Grafany.
2. Pokazanie aktualnych harmonogramów.
3. Wykonanie zapytania przez MCP z prośbą o wylistowanie wszystkich członków zespołów oraz ich przynależności do poszczególnych zespołów.
4. Uruchomienie Claude Code w odpowiednim folderze oraz wpisanie pierwszego polecenia konfiguracyjnego:

W trakcie pracy w tym wątku wykorzystuj zarówno MCP Grafany, jak i instrukcje z pliku `serwer.py`. W celu odpowiedniego dobierania narzędzi wykorzystuj umieszczony w tym projekcie skill.

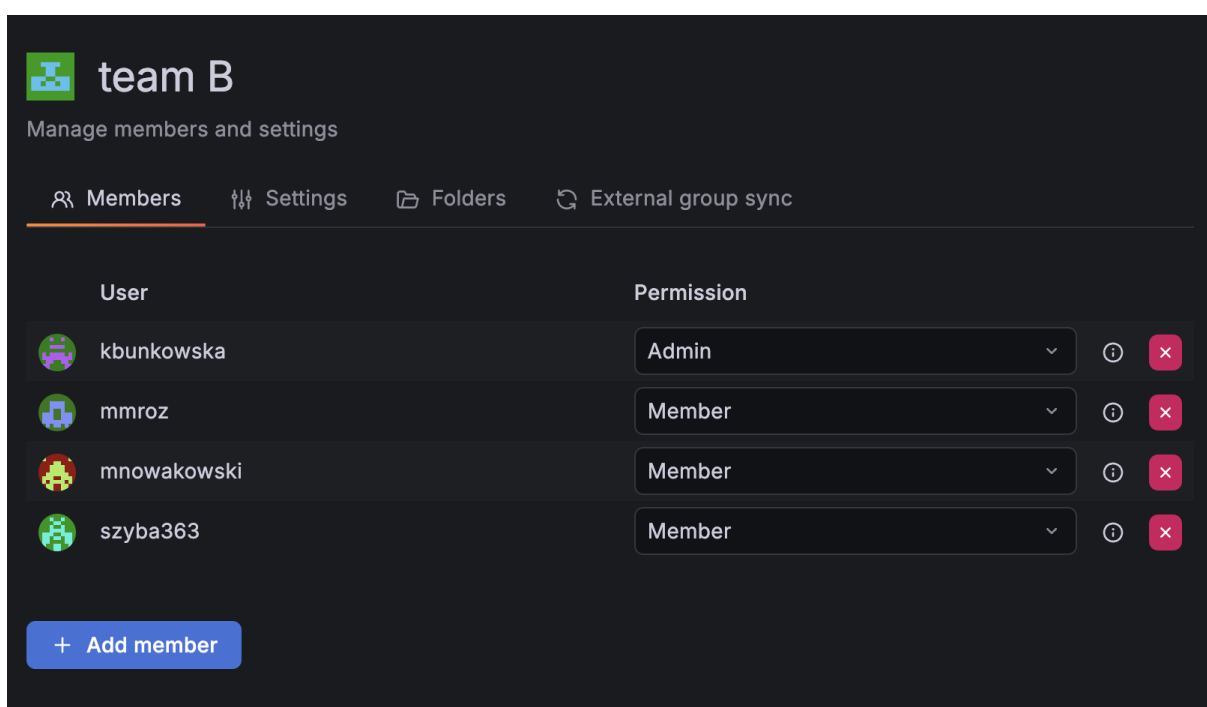
5. Poproszenie MCP o dodanie użytkownika mnowakowski do zespołu B.

Użyte polecenie:

Dodaj użytkownika mnowakowski do zespołu B.



Rysunek 9.1: Zespół B przed dodaniem użytkownika mnowakowski



Rysunek 9.2: Zespół B po dodaniu użytkownika mnowakowski

6. Poproszenie MCP o zmianę bazowej rotacji on-call w Grafanie.

Użyte polecenie:

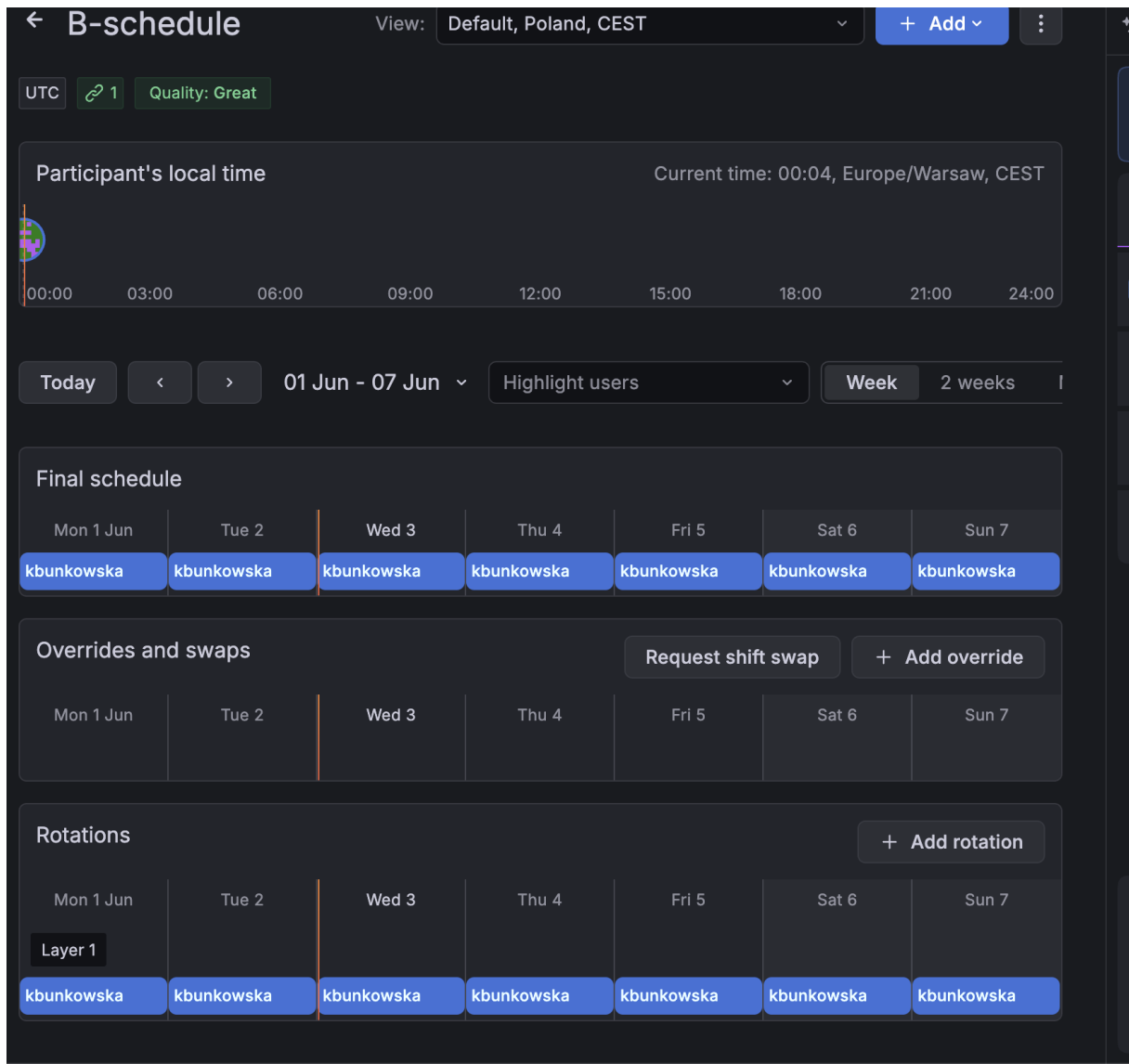
Podmień szyba363 na kbunkowska w bazowej rotacji B-schedule.

Final schedule						
Mon 1 Jun	Tue 2	Wed 3	Thu 4	Fri 5	Sat 6	Sun 7
szyba363	szyba363	szyba363	szyba363	szyba363	szyba363	szyba363

Overrides and swaps				Request shift swap		+ Add override
Mon 1 Jun	Tue 2	Wed 3	Thu 4	Fri 5	Sat 6	Sun 7

Rotations						+ Add rotation
Mon 1 Jun	Tue 2	Wed 3	Thu 4	Fri 5	Sat 6	Sun 7
Layer 1						
szyba363	szyba363	szyba363	szyba363	szyba363	szyba363	szyba363

Rysunek 9.3: Bazowa rotacja przed zmianą



Rysunek 9.4: Bazowa rotacja po zmianie użytkownika on-call

7. Wygenerowanie kilku przykładowych błędów, które zostaną wysłane do użytkownicy kbunkowska na telefon oraz na Discorda.
8. Poproszenie MCP o utworzenie wniosku o zastąpienie użytkownicy kbunkowska w harmonogramie w przyszłym tygodniu.

Użyte polecenie:

Utwórz dla użytkownicy kbunkowska prośbę o zmianę shiftu w najbliższą sobotę.

Final schedule						
Mon 1 Jun	Tue 2	Wed 3	Thu 4	Fri 5	Sat 6	Sun 7
szyba363	szyba363	szyba363	szyba363	szyba363	szyba363	szyba363

Overrides and swaps						
				Request shift swap	+ Add override	
Mon 1 Jun	Tue 2	Wed 3	Thu 4	Fri 5	Sat 6	Sun 7
Swaps						szyba363 → ?

Rysunek 9.5: Przykładowa prośba o zmianę shiftu utworzona przez MCP dla użytkownika szyba363

9. Poproszenie MCP o zaprezentowanie podsumowania aktywnych i rozwiązanych alertów z ostatnich dwóch tygodni.

Użyte polecenie:

Zwróć mi wszystkie alerty z ostatniego miesiąca z podziałem na te dalej aktywne i te już rozwiązane.

● ---

Alerty z ostatniego miesiąca (2026-05-02 → 2026-06-02)

● **Aktywne (1)**

#	Nazwa	Od	Alerty
1	BadRequest_error (istio)	2026-05-23 17:10 UTC	1

| Aktywny od 10 dni – bez rozwiązania.

✓ **Rozwiązane (30)**

2026-05-31 – 4 alerty (wszystkie P99 latency, ~10 min):

#	Nazwa	Czas trwania
2	P99_filter (frontend, istio)	10 min
3	P99_filter (frontend-external, istio)	10 min
4	P99_filter (paymentservice:50051, istio)	10 min
5	P99_filter (checkoutservice, istio)	10 min

2026-05-23 – 26 alertów (intensywny dzień testów i incydentów):

#	Nazwa	Czas trwania
6	Test Alert (team A)	~20h (do 24 maja)
7–12	Test Alert (team A) × 5	3–17 min
13	BadRequest_error (istio)	3 min
14–16	P99_filter (różne serwisy)	5–10 min
17–21	P99_filter / DatasourceNoData / TestData	2–14 min
22–31	TestAlert, InstanceDown, (bez tytułu) × 3	1–180 min

Uwaga: alert `BadRequest_error (istio)` z poz. 1 jest nadal aktywny od 10 dni – warto sprawdzić.

Rysunek 9.6: Raport alertów zwrócony przez agenta

Rozdział 10

Podsumowanie i wnioski

W ramach projektu udało się zbudować i skonfigurować środowisko do monitorowania oraz zarządzania incydentami oparte na platformie Grafana. Zintegrowano aplikację mikroservisową z narzędziami zbierającymi dane telemetryczne, które posłużyły do konfiguracji reguł alertowania oraz stworzenia systemu powiadamiania o incydentach. Opracowano również serwer MCP umożliwiający konfigurację kluczowych elementów obsługi incydentów, takich jak harmonogramy dyżurów. W trakcie demonstracji przeprowadzono symulację awarii aplikacji, a następnie przy wykorzystaniu serwera MCP modyfikowano konfigurację środowiska i obserwowano działanie mechanizmów Grafana IRM odpowiedzialnych za generowanie oraz dystrybucję alertów.

Całość projektu można uznać za udaną. Z powodzeniem wykorzystano protokół MCP do uproszczenia konfiguracji środowiska zarządzania incydentami przy użyciu języka naturalnego. Zastosowane rozwiązanie pokazało potencjał integracji modeli językowych z narzędziami operacyjnymi, natomiast Grafana IRM potwierdziła swoją skuteczność w zakresie wykrywania incydentów, zarządzania powiadomieniami oraz koordynacji procesu reagowania na awarie.